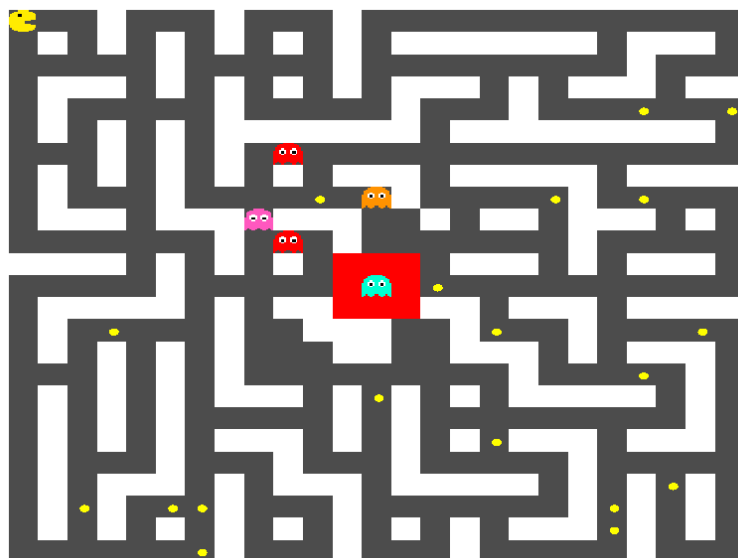


Université Marie & Louis Pasteur
UFR Sciences et Techniques

Projet L3 : Pacman en réseau

Rapport de Projet

Licence CMI Informatique - 3ème année
2025-2026



Thomas COUTANT
Benoît LITAMPHA
Auriane PETER

Encadrant : Julien BERNARD

Remerciements

Table des matières

Remerciements	1
Introduction	5
1 Présentation du projet	5
1.1 Présentation générale du jeu Pac-Man	5
1.2 Contexte académique	6
1.3 Objectifs généraux du projet	6
2 Définition du sujet et problématique	7
2.1 Problématique du jeu en réseau	7
2.2 Choix techniques	7
2.3 Répartition des rôles	7
2.4 Conditions de victoire	8
2.5 Pac-gommes spéciales	8
2.6 Asymétrie de vision	8
2.7 Cohérence du système	8
3 Structure du serveur	9
3.1 Architecture multithread	9
3.2 Architecture modulaire	10
3.3 Boucle de jeu	11
3.4 Plateau et génération procédurale	12
3.5 Intelligence artificielle des fantômes	16
3.6 Rooms : gestion des parties	18
4 Structure du client	21
4.1 Bases et Objectifs du Client	21
4.2 Architecture logicielle et SceneManager	21
4.3 Scènes et Entités	22
4.4 Réseau coté Client : réception	23
4.5 Réseau coté Client : envois	23
4.6 Input, Actions, et EventTypes	24
4.7 Rendu des Entités	24
5 Protocoles et échanges réseau	26
5.1 Type de communication	26
5.2 Format des messages	26
5.3 Protocole	27
6 Bilan et perspectives	31
6.1 Compétences acquises	31
6.2 Résultat final	31
6.3 Améliorations possibles	31
Conclusion	32
Sitographie	33

Résumé	34
Mots-Clés	34
Abstract	34
Key-Words	34

Table des figures

1	PacMan : le jeu original	5
2	Exemple de Chaîne de responsabilité	7
3	Chaîne de responsabilité côté serveur	10
4	Diagrammes de classe de l'architecture réseau	11
5	Plateau après l'algorithme de Prim	13
6	Plateau après génération de la hut	14
7	Plateau après ajout de cycles	15
8	Plateau après suppression des culs-de-sac et ajout des portails	16
9	Vue des traces	18
10	Vue de la Room	20
11	Échanges entre le client et le serveur lors de la connexion/lobby	28
12	Échanges entre le client et le serveur lors dans le salon/en jeu	30

Introduction

1 Présentation du projet

1.1 Présentation générale du jeu Pac-Man

Le jeu **Pac-Man**, apparu au début des années 1980, est un jeu d'arcade dans lequel le joueur contrôle un personnage évoluant dans un labyrinthe. L'objectif principal est de parcourir l'ensemble du labyrinthe afin de consommer toutes les **pac-gommes** qui s'y trouvent, tout en évitant d'entrer en contact avec des fantômes. Ces derniers constituent les adversaires du joueur et cherchent à bloquer ou intercepter ses déplacements. Certaines **pac-gommes** spéciales permettent temporairement à **Pac-Man** d'inverser les rôles en devenant capable de manger les fantômes, ajoutant ainsi une dimension stratégique au gameplay.

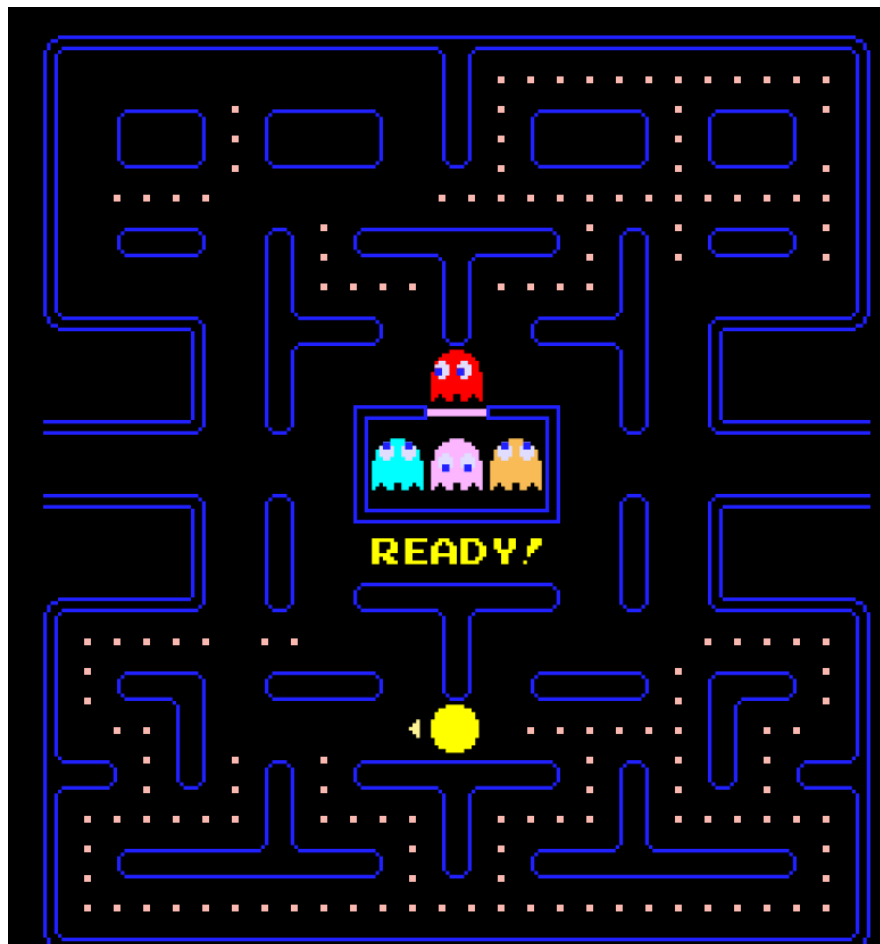


FIGURE 1 – PacMan : le jeu original

Dans le cadre de ce projet, le jeu a été adapté pour le multijoueur. Une partie oppose un joueur **Pac-Man** à plusieurs joueurs fantômes. Lorsque le nombre de joueurs est insuffisant, certains fantômes sont contrôlés par des intelligences artificielles afin de ne pas déséquilibrer la partie.

Le choix de **Pac-Man** comme sujet pour une version en réseau est stratégique car ses règles simples et très connues, ce qui permet une facilité d'implémentation par rapport à d'autres jeux. De plus, le gameplay se prête à une séparation des rôles entre plusieurs joueurs.

1.2 Contexte académique

Ce projet a été réalisé dans le cadre du semestre six de la licence informatique de l'Université Marie & Louis Pasteur. Il s'inscrit dans un enseignement visant à mettre en pratique nos connaissances à travers la réalisation d'un gros projet à plusieurs.

Le travail a été mené en groupe de trois étudiants de mi-octobre à la fin du mois de mars, sous l'encadrement de Julien Bernard, maître de conférences à l'Université Marie & Louis Pasteur. Le projet porte sur le développement d'un jeu vidéo en réseau.

Les objectifs de ce projet sont de consolider nos compétences en programmation C++, approfondir la découverte du développement de jeux vidéo, ainsi qu'à mettre en place une architecture réseau client-serveur. Le projet permet également de développer des compétences telles que le travail en équipe et la gestion d'un projet informatique sur une longue période.

1.3 Objectifs généraux du projet

Les objectifs du projet peuvent être divisés en deux catégories :

- les objectifs fonctionnels, liés à l'expérience de jeu
- les objectifs techniques, liés à la conception et à l'implémentation du système

Du point de vue fonctionnel, l'objectif était de concevoir un jeu Pac-Man jouable en réseau qui permet à plusieurs joueurs de participer à une même partie.

Sur le plan technique, le projet visait à mettre en place une architecture réseau client-serveur. Le serveur devait être capable de gérer les connexions des joueurs, la création et la gestion des parties, ainsi que l'échange des données nécessaires au jeu.

2 Définition du sujet et problématique

2.1 Problématique du jeu en réseau

La conception d'un jeu multijoueur en temps réel implique plusieurs contraintes. La difficulté majeure concerne la synchronisation des états du jeu entre les clients. Chaque joueur doit avoir une vision cohérente de l'ensemble de la partie (les positions de Pac-Man, des fantômes et l'état des pac-gommes, ect). Pour garantir cette cohérence, nous avons opté pour un serveur autoritaire : toutes les décisions sont validées côté serveur, qui diffuse ensuite les informations à tous les clients. Qui, de leur côté, envoient uniquement leurs intentions de déplacement, qui sont ensuite traitées et synchronisées par le serveur. Les données transitent via des buffers et des files avant d'être appliquées. Cela permet un traitement ordonné des événements du jeu.

2.2 Choix techniques

Le projet devait être développé en C++ en utilisant la bibliothèque *Gamedev Framework* (gf), qui est adaptée au développement de jeux vidéo et est compatible avec l'architecture réseau. La communication réseau repose sur le protocole TCP, assurant la sûreté des échanges entre le serveur et les clients. Les messages sont sérialisés afin de structurer les informations transmises.

Nous avons par contre choisi de concevoir le serveur selon un modèle multithread avec une chaîne de responsabilité pour gérer les packets, reflétant la logique de fonctionnement du jeu : un thread¹ principal gère le serveur et le lobby, tandis que d'autres threads s'occupent des rooms, de la boucle de jeu, des fantômes contrôlés par l'IA et de la communication avec les clients. Cette architecture permet de traiter simultanément plusieurs interactions des joueurs et d'assurer une exécution fluide et cohérente des parties.

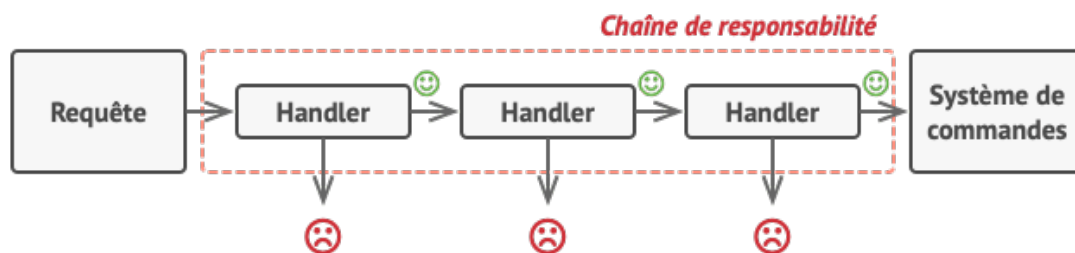


FIGURE 2 – Exemple de Chaîne de responsabilité

2.3 Répartition des rôles

La réalisation de ce projet a été organisée de manière collaborative entre les trois membres du groupe.

Thomas Coutant a principalement travaillé sur le développement du serveur, ainsi que sur la conception de la logique du jeu. Il a également développé un prototype

1. Un thread (fil d'exécution) est une unité d'exécution légère au sein d'un processus. Dans une architecture client-serveur, il permet par exemple de gérer plusieurs connexions ou tâches simultanément (réception réseau, logique de jeu, envoi des mises à jour) tout en partageant la même mémoire.

minimal de démarrage, permettant de tester la communication client–serveur avec un simple cube représentant un joueur en mouvement.

Auriane Peter s’est concentrée sur le développement du client, en implémentant l’interface et la gestion des interactions côté joueur.

Benoît Litampha a assuré la communication entre le serveur et le client, participant également à certaines tâches sur le serveur et le client lorsque nécessaire.

Cette répartition a permis une progression simultanée sur plusieurs fronts du projet, tout en gardant une cohérence globale de l’application.

Les règles du jeu ont été définies afin de mettre en place un équilibre entre tension et stratégie. Chaque camp dispose d’objectifs et contraintes différents, créant une structure de compétition.

2.4 Conditions de victoire

Pac-Man gagne la partie s’il parvient à manger tout les pac-gommes du plateau ou s’il survit jusqu’à la fin du timer. Les fantômes, eux, gagnent en épuisant toutes les vies de Pac-Man avant la fin du temps imparti.

Ce double système de victoire met une pression permanente : Pac-Man doit optimiser ses déplacements, tandis que les fantômes doivent coordonner leurs actions pour limiter ses déplacements.

2.5 Pac-gommes spéciales

Certaines pac-gommes déclenchent un mode temporaire appelé « chasseur ». Durant cette période, Pac-Man devient capable de manger les fantômes. Cet état est limité dans le temps par un timer. Les fantômes éliminés réapparaissent mais ont aussi des vies, maintenant ainsi l’équilibre de jeu car les pac-gommes spéciales sont peu nombreuses.

Ce mécanisme introduit un changement brutal du gameplay, les chasseurs deviennent les chassés et inversement.

2.6 Asymétrie de vision

Le système repose également sur une différence de vision. Pac-Man a une vue globale du plateau, tandis que les fantômes ne voient uniquement environnement limités, ils ne peuvent voir qu’à une certaine distance d’eux même, la vision des fantômes n’est pas partagée entre eux.

2.7 Cohérence du système

La combinaison de ces règles avec la génération procédurale du plateau et l’architecture hybride de l’intelligence artificielle et des joueurs, produit un gameplay varié et imprévisible. Les interactions entre le timer, le manque de vision et inversement temporaire des rôles permettent de nombreuses stratégies et augmentent la rejouabilité du jeu.

3 Structure du serveur

3.1 Architecture multithread

Le serveur repose sur une architecture multithread² conçue pour supporter plusieurs connexions simultanées sans dégradation des performances. La couche réseau est strictement séparée de la logique de jeu afin d'éviter qu'une opération bloquante, attente de paquet³, latence élevée ou déconnexion, n'impacte l'exécution globale du système.

Un thread est dédié à la réception des paquets réseau. Les messages entrants sont insérés dans une file d'attente interne, puis traités de manière synchronisée par la logique centrale du serveur. Cette organisation assure une séparation claire entre la communication et la logique.

La file de messages agit comme un tampon entre le réseau et la boucle de jeu à tick⁴ fixe. Ainsi, les mises à jour périodiques du monde restent indépendantes des variations de latence ou du débit des clients.

Cette architecture permet l'exécution simultanée de plusieurs parties indépendantes au sein d'un même serveur.

2. Un programme multithread exécute plusieurs fils d'exécution (threads) en parallèle au sein d'un même processus.

3. Un paquet réseau est une unité de données formatée et structurée, transmise à travers un réseau. Il contient généralement une en-tête (header) avec des informations de routage et de contrôle, ainsi qu'une charge utile (payload) correspondant aux données transportées.

4. Un tick désigne une unité de temps discrète utilisée par le moteur du jeu pour mettre à jour l'état du monde. À chaque tick, le serveur traite les entrées des joueurs, met à jour la logique de jeu (physique, collisions, scores, etc.) puis envoie les nouvelles informations aux clients.

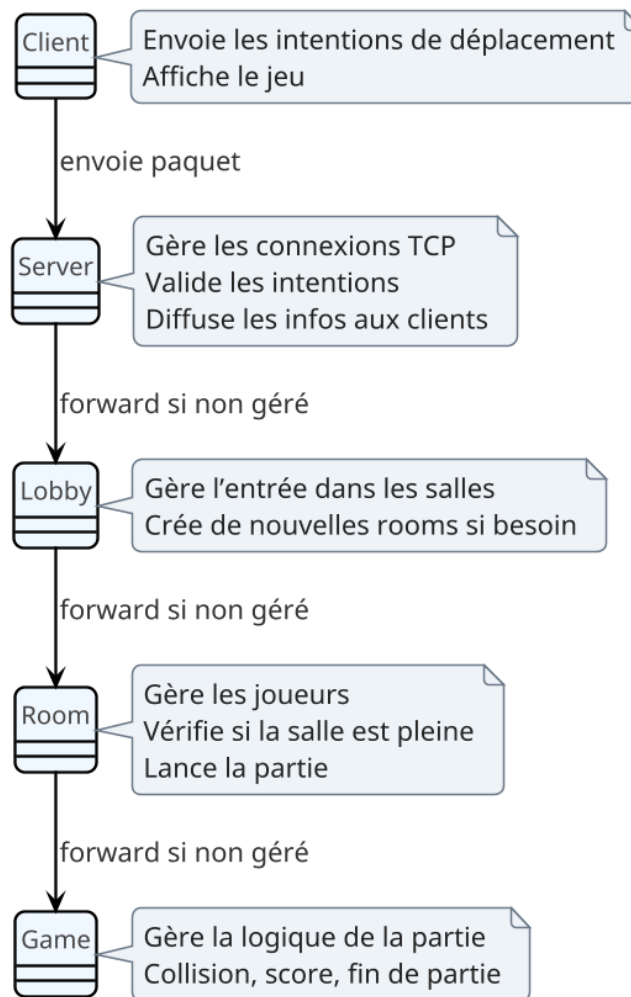


FIGURE 3 – Chaîne de responsabilité côté serveur

3.2 Architecture modulaire

L'architecture du serveur est organisée en plusieurs couches distinctes possédant chacune une responsabilité. Cette structuration modulaire garantit lisibilité, extensibilité et maintenabilité.

La couche *Server* assure la gestion des connexions et le routage des messages. Le *Lobby* centralise l'organisation des joueurs et la création des parties. Chaque partie est enfermée dans une entité *Room*, responsable de son état propre et de ses joueurs. Enfin, la logique complète du jeu est contenue dans la classe *Game*, indépendante des mécanismes réseau.

L'architecture suit ainsi une chaîne de responsabilité hiérarchique :

Client → Server → Lobby → Room → Game

Chaque couche délègue les traitements spécialisés au niveau inférieur, ce qui permet d'introduire de nouvelles fonctionnalités sans modifier les composants fondamentaux du système.

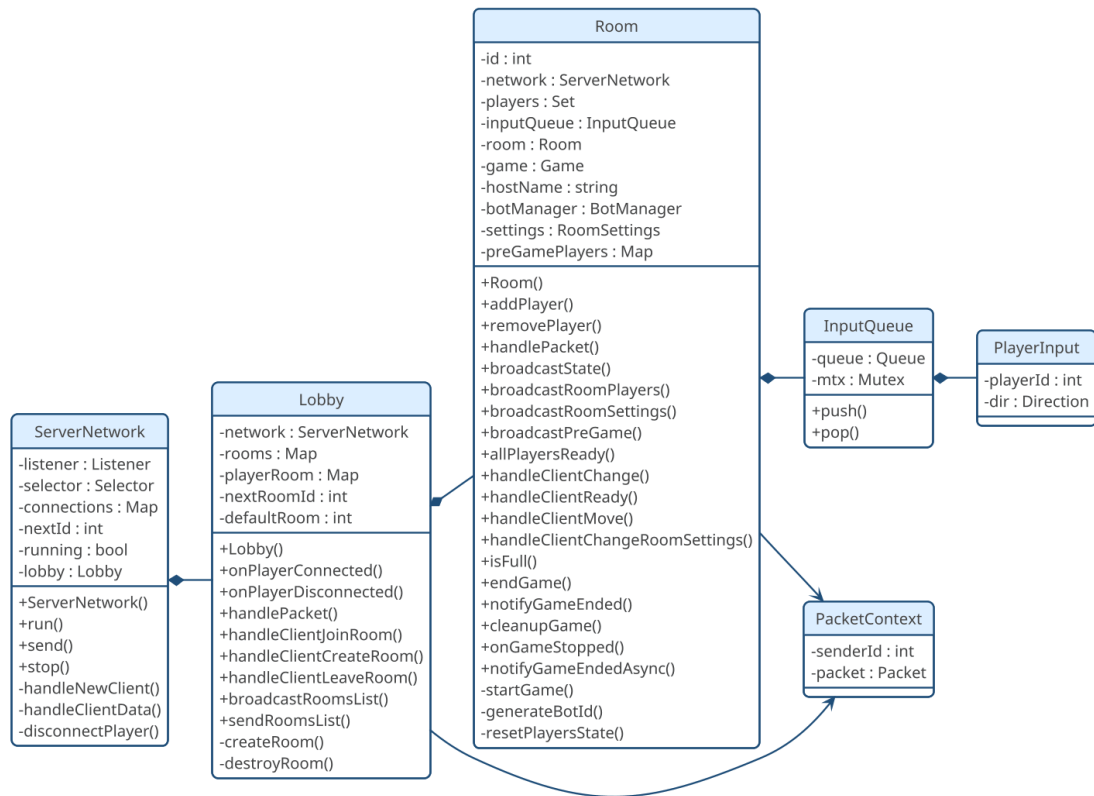


FIGURE 4 – Diagrammes de classe de l'architecture réseau

Vue côté serveur - Gestion des packets, des parties, des salles (sans geter et seter)

3.3 Boucle de jeu

La boucle de jeu constitue le cœur du serveur et repose sur un modèle à mise à jour périodique, appelé *tick serveur*. Contrairement à une architecture purement événementielle, l'état du monde est recalculé à intervalle régulier selon une fréquence fixe. Ce fonctionnement garantit un comportement déterministe et centralisé.

À chaque tick, le serveur exécute une séquence d'opérations. Les intentions de déplacement reçues des clients sont d'abord traitées, puis les positions des entités sont mises à jour en fonction des règles. Les états temporaires sont ensuite décrémentés, le score est ajusté et les conditions de fin de partie sont vérifiées.

Les rôles sont attribués au début de chaque partie : un joueur incarne Pac-Man tandis que les autres incarnent les fantômes. Chaque rôle possède des contraintes qui influencent les déplacements et les interactions. L'ensemble de ces règles est appliqué exclusivement côté serveur, il est le seul qui décide.

Le plateau contient deux types de pac-gommes. Les pac-gommes classiques augmentent le score lors de leur consommation, tandis que les pac-gommes spéciales déclenchent un mode temporaire dit « chasseur ». Ce mode est géré par un système de minuterie interne décrémenté. Lorsque ce compteur atteint zéro, l'état normal est restauré.

Il y a 2 timers, un timer global définissant la durée maximale d'une partie, et un timer local associé aux états temporaires. Cette gestion du temps garantit une certaine cohérence entre les événements et permet l'indépendance des événements du jeu.

Les collisions sont évaluées avant chaque mise à jour de position. Un déplacement vers une case murale est annulé ou sur une case hut pour **PacMan**. Lors d'une collision entre Pac-Man et un fantôme, l'issue dépend de l'état courant : perte d'une vie en mode normal, perte d'une vie du fantôme en mode chasseur. L'intégralité de ces calculs étant effectuée côté serveur, toute tentative de triche ou de désynchronisation est neutralisée.

À la fin de chaque tick, l'état global mis à jour est diffusé à l'ensemble des clients. Ces derniers ne réalisent aucun calcul logique : ils se contentent d'afficher l'état reçu. Cette architecture s'inscrit dans le modèle de serveur autoritaire défini précédemment.

3.4 Plateau et génération procédurale

Le plateau repose sur une génération procédurale de labyrinthe fondée sur une variante de l'algorithme de Prim. Ce choix permet de produire automatiquement des cartes connexes avec une forte variabilité entre les parties.

La génération débute par un remplissage complet du plateau avec des murs. Une case de départ est ensuite sélectionnée et marquée comme traversable. Ses murs adjacents sont ajoutés à une liste dite *frontier*. À chaque itération, un mur est choisi aléatoirement dans cette liste. S'il sépare une case déjà accessible d'une case encore murale, il est supprimé, la nouvelle case devient traversable, et ses propres murs sont ajoutés à la *frontier*. Le processus se poursuit jusqu'à épuisement de cette liste.

Ce mécanisme produit un arbre couvrant avec un labyrinthe connexe sans zones isolées.

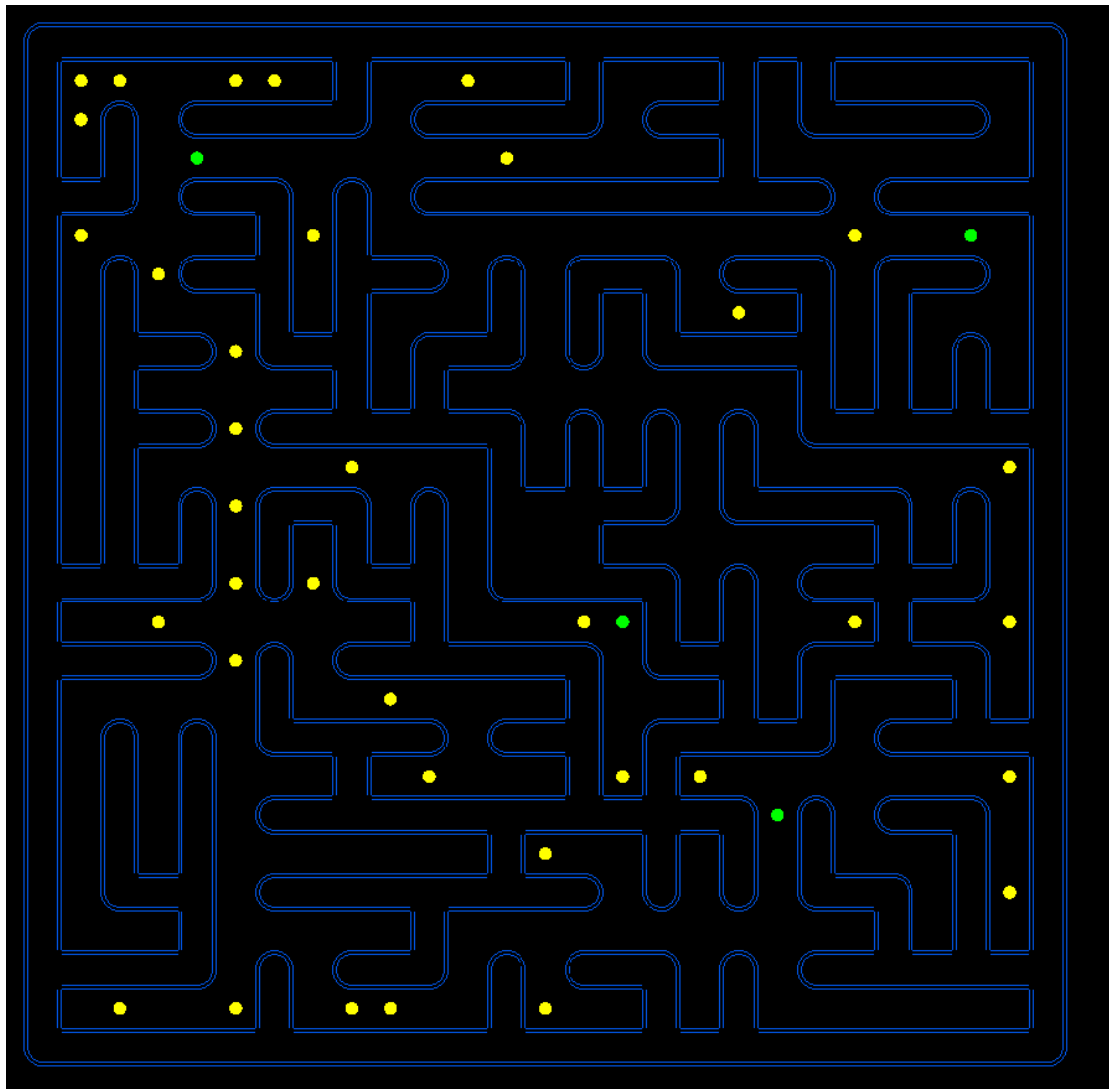


FIGURE 5 – Plateau après l'algorithme de Prim

Afin d'adapter cette structure aux contraintes du gameplay, plusieurs transformations supplémentaires sont appliquées.

Une zone centrale fixe de taille 3×3 , appelée « hut », est réservée avant la génération. Elle est protégée durant l'exécution de l'algorithme afin de préserver sa structure. Une fonction dédiée vérifie ensuite son raccordement au labyrinthe principal par au moins une ouverture.

Les coins du plateau, destinés aux positions initiales des joueurs, sont également vérifiés après la génération. Si l'un d'eux se retrouve isolé, une ouverture contrôlée est créée pour garantir son accessibilité.

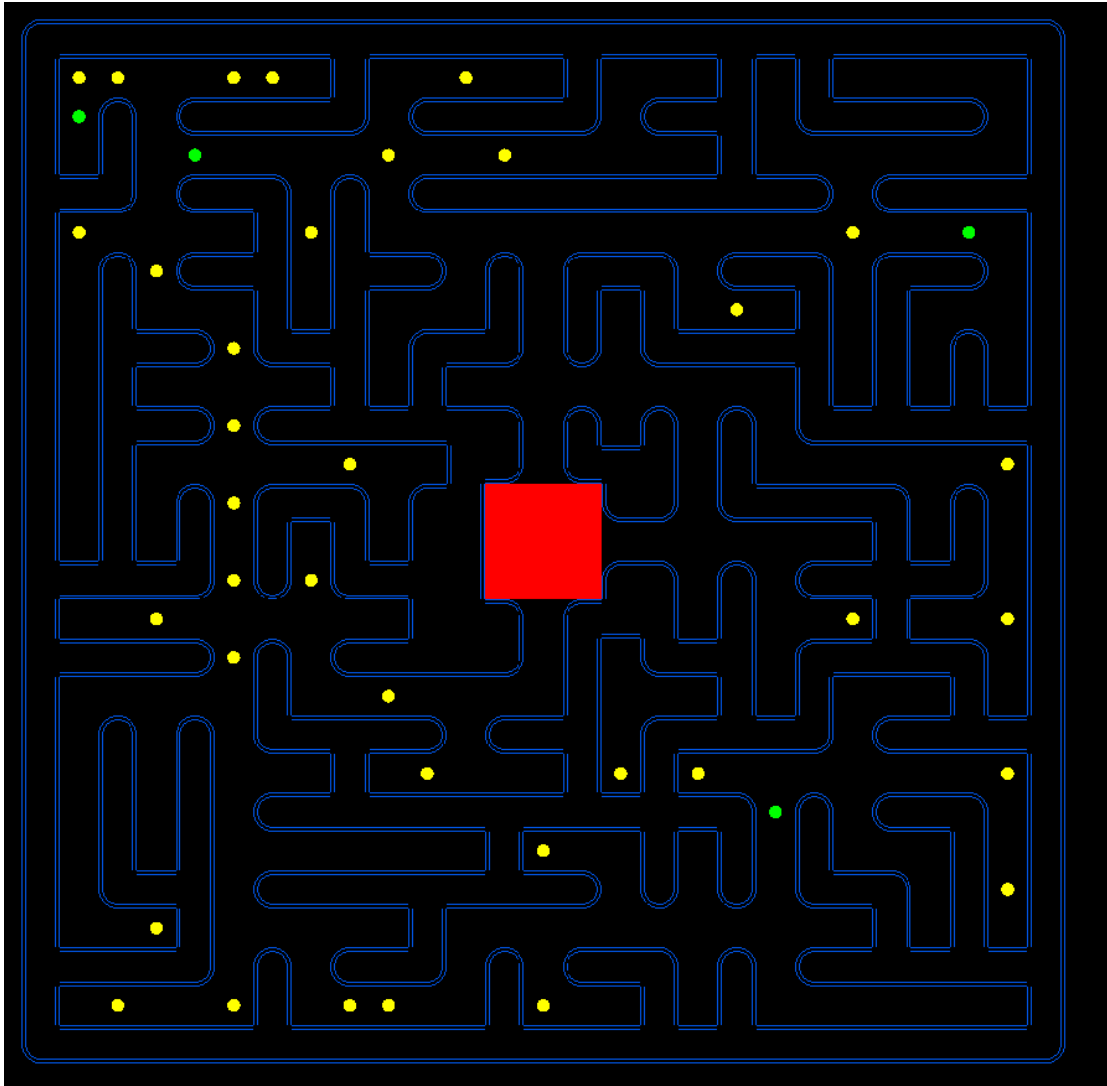


FIGURE 6 – Plateau après génération de la hut

L'algorithme de Prim produisant un arbre sans cycles, des ouvertures supplémentaires sont ajoutées de manière aléatoire afin d'ajouter des chemins. Cette étape améliore la poursuite par les fantômes et évite des trajectoires trop prévisibles.

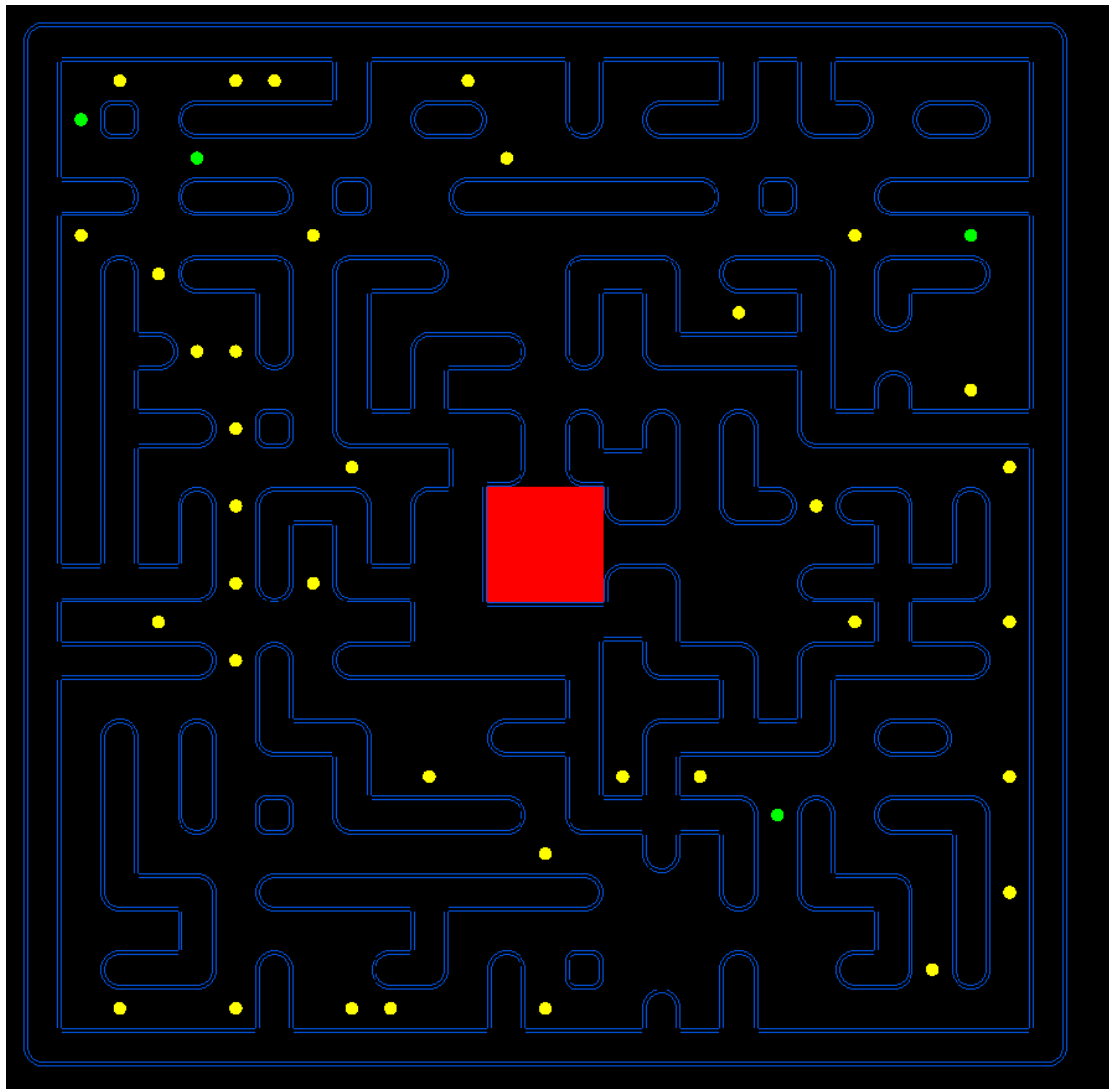


FIGURE 7 – Plateau après ajout de cycles

Enfin, certains culs-de-sac sont supprimés en identifiant les cases ne possédant qu'un unique voisin accessible. La suppression partielle de ces impasses améliore la fluidité des déplacements et limite les situations punitives.

Des portails latéraux sont également intégrés sur les bords du plateau, ce qui permet à une entité de réapparaître du côté opposé.

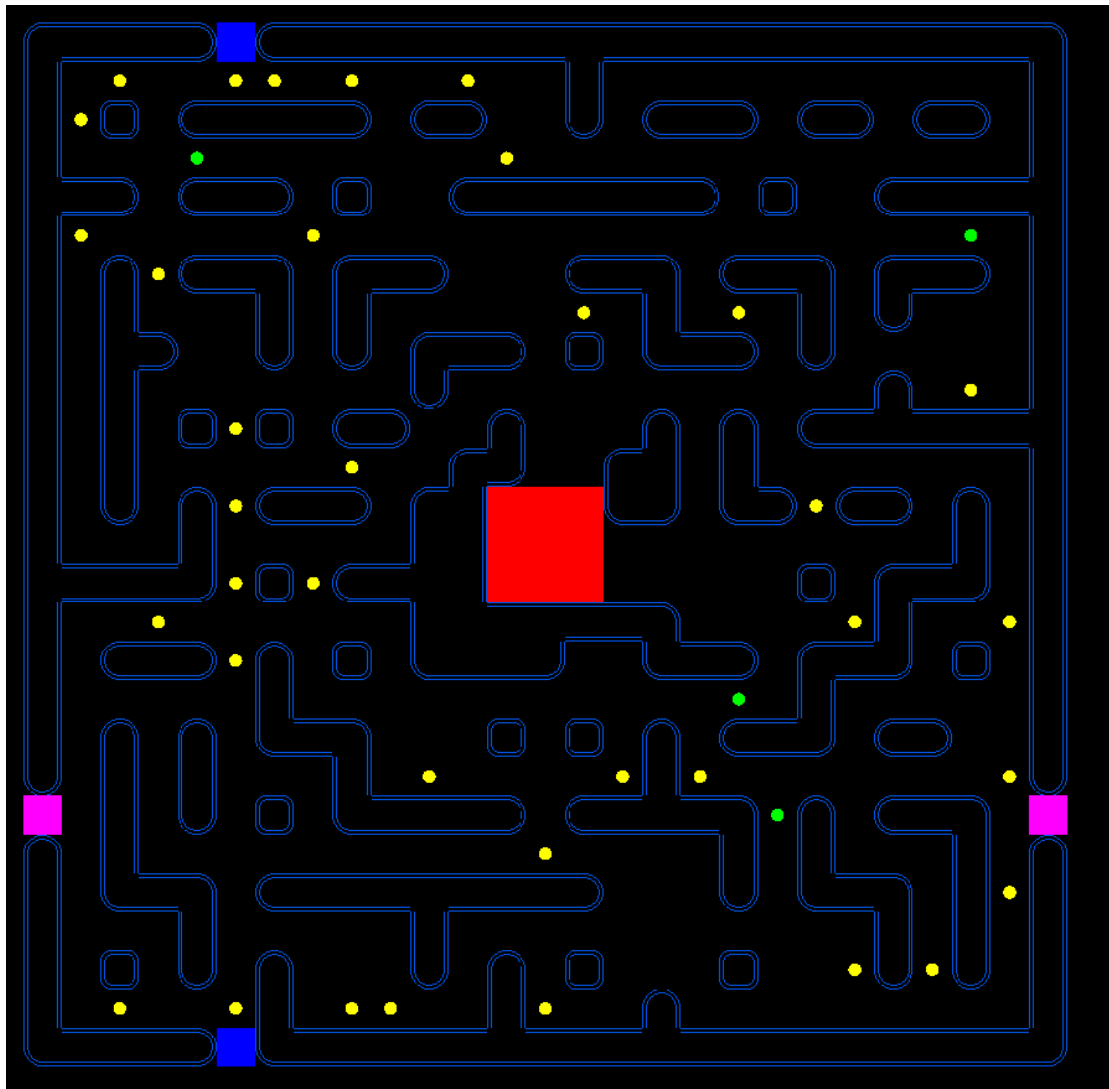


FIGURE 8 – Plateau après suppression des culs-de-sac et ajout des portails

Le plateau est une structure centrale du système : il sert de support aux collisions, au placement des pac-gommes, ainsi qu'aux algorithmes de poursuite des fantômes. La génération procédurale contribue à la rejouabilité et à la diversité stratégique des parties.

3.5 Intelligence artificielle des fantômes

L'intelligence artificielle des fantômes repose sur une architecture hybride qui mélange pathfinding classique, système d'influence local et prise de décision hiérarchique. L'objectif n'est pas uniquement de poursuivre efficacement Pac-Man, mais de produire une intelligence artificielle adaptable et collective.

Architecture décisionnelle hiérarchique À chaque tick serveur, chaque non joueur évalue sa situation selon un ordre de priorité.

Lorsque Pac-Man est en mode chasseur, la priorité absolue devient la fuite stratégique vers la hut. Le plus court chemin est alors calculé à l'aide d'un algorithme

de Breadth First Search (BFS), garantissant une trajectoire optimale dans un graphe non pondéré. Ce choix permet la compatibilité avec les labyrinthes générés procéduralement.

En situation normale, si Pac-Man est détecté dans un rayon de vision défini, le fantôme active un mode de poursuite. Un BFS à profondeur limitée est exécuté afin de simuler une perception réduite de l'environnement. Cette limitation réduit le coût tout en conservant un comportement crédible et conforme aux règles du jeu.

En l'absence de menace ou de cible visible, le fantôme adopte un comportement autonome fondé sur un système de trace.

Système de trace Chaque case du plateau possède un score dynamique défini par :

$$Score = Attraction - Répulsions$$

Les traces laissées par Pac-Man génèrent une force attractive, tandis que la présence d'autres fantômes induit des forces répulsives. Le fantôme sélectionne localement la case voisine maximisant ce score.

Ce mécanisme s'inspire des modèles de colonies de fourmis, où l'information est distribuée dans l'environnement. Cela permet de faire une forme d'intelligence collective peu coûteuse. Ce qui m'a donné l'idée d'appliquer ce système est une vidéo sur Youtube : "J'ai codé une simulation de 869 fourmis", de Code Boy [2]

Comportement émergent L'interaction entre attraction vers la cible et répulsion inter-bots produit des comportements collectifs sans coordination explicite. Les fantômes tendent naturellement à se disperser, à éviter les regroupements inutiles et à converger progressivement vers Pac-Man en exploitant différentes trajectoires.

Choix algorithmiques L'implémentation combine plusieurs techniques complémentaires : le BFS pour le calcul de chemins optimaux, une version limitée pour la simulation de vision, une recherche locale pour l'exploration autonome. Un bruit aléatoire est également introduit afin d'éviter les cycles déterministes et les oscillations.

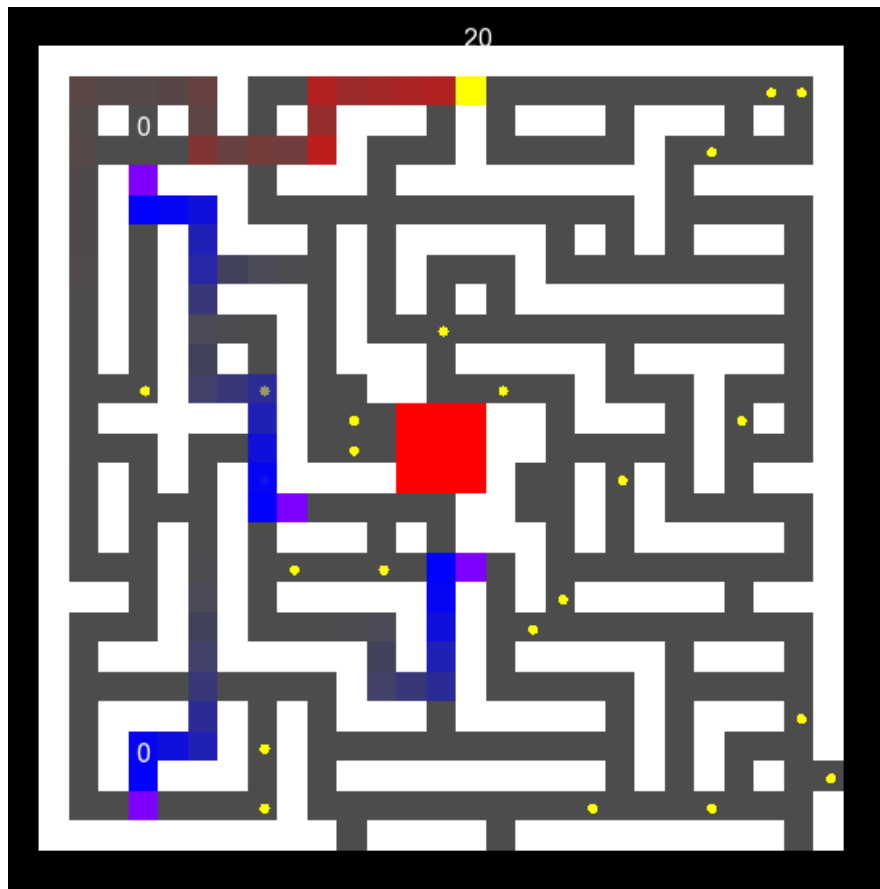


FIGURE 9 – Vue des traces

Rouge pour PacMan et Bleu pour les fantômes

3.6 Rooms : gestion des parties

Afin de supporter plusieurs parties simultanées, le serveur repose sur un système de *rooms*. Une room représente une instance autonome de jeu possédant son propre état, ses propres joueurs et son propre cycle d'exécution. Cette abstraction permet d'isoler complètement les parties les unes des autres tout en gardant une architecture serveur centralisée.

Chaque room maintient la liste des joueurs connectés, l'attribution des rôles (Pac-Man ou fantômes), les paramètres de la partie ainsi que son état courant (en attente, en cours, terminée). Elle encapsule également sa propre boucle de jeu et son plateau, garantissant qu'aucune donnée n'est partagée entre deux parties distinctes.

Création dynamique et attribution des rôles Lorsqu'un joueur rejoint le lobby, il est dirigé vers une room existante disposant de places libres. Si aucune room n'est disponible, une nouvelle room est créée dynamiquement. Cette création à la demande permet au serveur d'adapter automatiquement sa structure au nombre de joueurs connectés.

L'attribution des rôles est effectuée au moment du lancement de la partie selon une règle simple : un joueur incarne Pac-Man tandis que les autres deviennent des fantômes. Ce mécanisme est géré exclusivement côté serveur afin d'éviter toute incohérence ou

manipulation côté client. En cas de conflit le server informe les clients de la règle non respectée.

Gestion et diffusion de l'état Chaque room possède un état centralisé comprenant les positions des joueurs, l'état des pac-gommes et les différents systèmes temporels. À chaque tick, cet état est mis à jour puis diffusé uniquement aux joueurs appartenant à la room concernée.

Ce cloisonnement garantit la cohérence interne de chaque partie et empêche toute interférence.

Paramétrisation des parties Les rooms permettent d'ajuster dynamiquement certains paramètres, tels que la durée maximale d'une partie, le nombre de vies de Pac-Man, ect. Cette capacité de configuration facilite les phases de test, l'équilibrage du gameplay et l'organisation de parties avec des règles customisées.

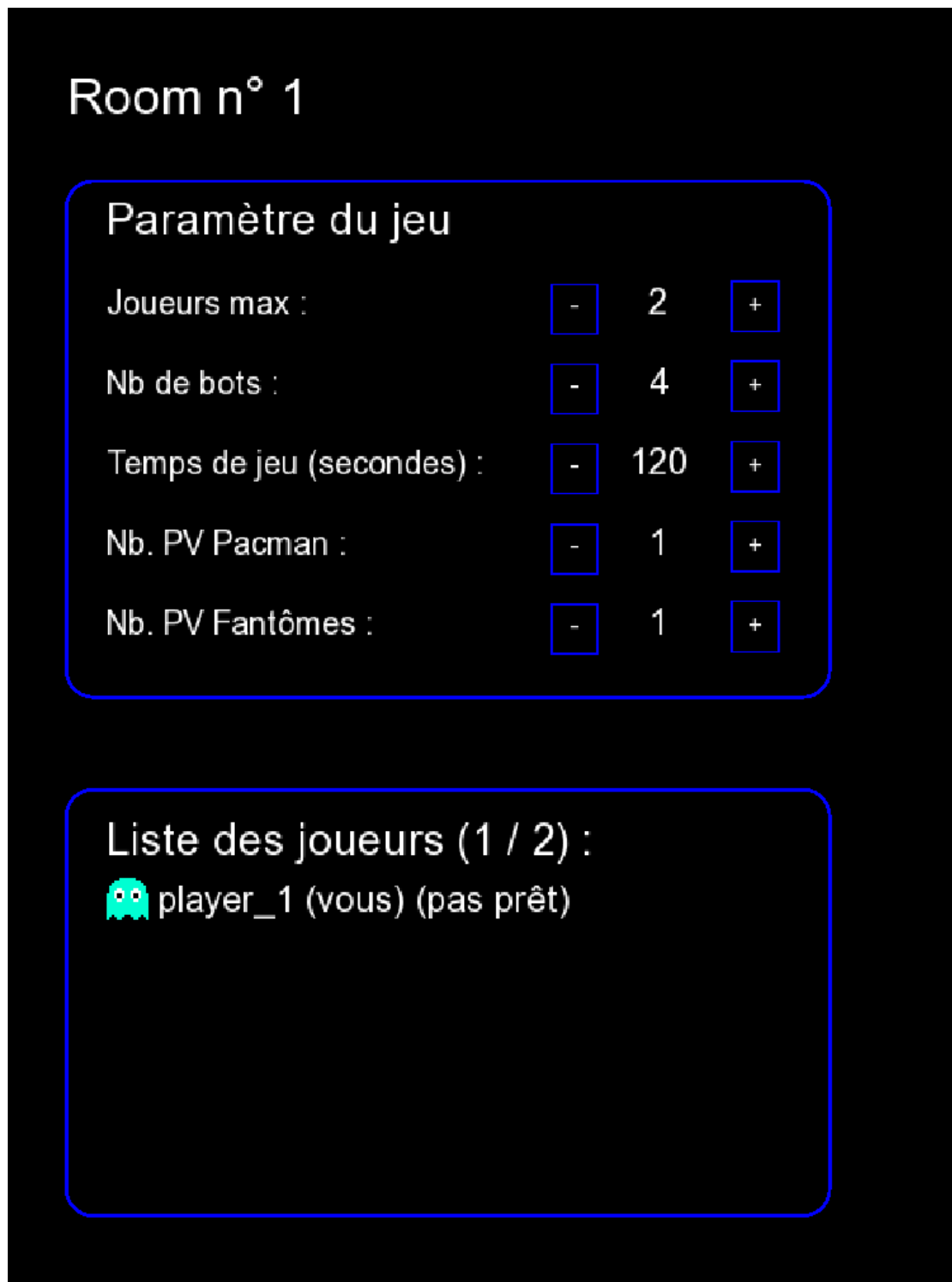


FIGURE 10 – Vue de la Room

4 Structure du client

4.1 Bases et Objectifs du Client

Le client a pour rôle principal d'être l'interface entre l'utilisateur et le serveur. Il ne décide pas des règles du jeu ni de la validité des actions cela est le rôle du serveur : il capture les entrées du joueur, les transmet au serveur, reçoit les états calculés côté serveur, puis les transforme en représentation visuelle pour le joueur. Dans le cadre d'un Pac-Man multijoueur, le client doit gérer des contraintes spécifiques : latence réseau, cohérence des états, fluidité visuelle, transitions entre différentes phases de jeu et gestion d'une interface utilisateur complète (menu, lobby, partie, fin de jeu).

Le client doit également gérer l'ensemble du cycle de vie d'une session de jeu. Cela inclut une scène d'accueil, une scène présentant les règles afin de clarifier le fonctionnement de la version multijoueur de Pacman, une page de liste de lobbys, une scène de configuration de salle, la transition vers la partie, puis l'écran de fin affichant le résultat. Le client constitue ainsi une interface complète permettant au joueur de naviguer entre différents contextes sans jamais interagir directement avec la logique serveur.

Tout en accomplissant ces objectifs variés, le client doit maintenir une séparation nette entre les différents éléments nécessaires à ces objectifs comme l'acquisition des entrées, la communication réseau (réception et envoi) ainsi que le rendu graphique. En séparant proprement les différents éléments du client, une organisation modulaire est mise en place qui facilite les changements et les évolutions du projet.

4.2 Architecture logicielle et SceneManager

L'architecture du client repose sur une classe centrale `ClientGame` héritant de `gf : SceneManager`. Ce choix structure l'application autour d'un gestionnaire de scènes qui va à lui seul piloter le cycle de vie complet du jeu. Le `SceneManager` maintient une pile de scènes actives et délègue à la scène courante la gestion des événements, des mises à jour en fonction des packets réseau reçus et du rendu.

Chaque scène représente un contexte fonctionnel distinct : `WelcomeScene`, `LobbyListScene`, `LobbyScene`, `RulesScene`, `GameScene`, `EndScene`. Chacune encapsule sa propre logique, ses entités graphiques et ses traitements d'événements. Cette organisation permet une séparation nette des responsabilités : une scène gère son état, son affichage et ses interactions, sans dépendre d'un état global centralisé.

Le `SceneManager` permet de remplacer ou empiler des scènes via des méthodes comme `replaceAllScenes`, `pushScene` ou `popAllScenes`. Le cycle de vie devient explicite : on entre dans la scène, interagit avec elle, puis sortons proprement.

La base du programme reflète cette structuration. La fonction `main` ne contient aucune logique métier ni aucun traitement graphique. Elle instancie simplement `ClientGame`, récupère éventuellement les paramètres réseau (IP et port), puis appelle `run`. Ainsi, toute la logique applicative est déléguée à l'architecture interne du client. Le `main` devient un simple déclencheur, ce qui confirme que l'organisation repose entièrement sur le `SceneManager` et ses scènes.

Interaction avec le réseau et les actions Bien que Le `SceneManager` n'intègre pas directement la logique réseau, il permet aux scène d'exploiter proprement et indépendamment cette dernières. Le thread réseau alimente une file de paquets

partagée. Chaque scène peut consommer ces paquets selon ses besoins et envoie ses packets, en fonctions des actions réalisées par le joueur.

Les actions sont également gérées par leur scènes respectives, permettant une clarté vis-à-vis de leur utilité.

Comparaison brève avec l'architecture initiale La première implémentation centralisait l'ensemble du rendu dans un unique module `render.cc`. Une variable d'état globale déterminait quelle portion de code exécuter grâce à une structure conditionnelle de type "Si".

Cette solution fonctionnait pour un prototype, mais montrait rapidement ses limites. L'affichage, logique partielle et la gestion d'événements étaient centralisées dans un seul fichier. Tout le système réseau également. L'ajout d'une nouvelle scène impliquait de modifier un fichier déjà volumineux, ce qui réduisait grandement la lisibilité. Avec l'augmentation du nombre de scènes, la structure conditionnelle devenait difficile à maintenir.

Le passage au `SceneManager` apporte de nombreuses améliorations comme une séparation claire des responsabilités (chaque scène gère ses actions, interactions réseau et affichage), facilité d'ajout de nouvelles scènes, un cycle de vie géré par une seule entité et des transitions "automatiquement" gérées par les scènes.

4.3 Scènes et Entités

La conception du client repose sur une séparation claire entre la logique et la représentation visuelle. Cette distinction se matérialise par la relation entre `GameScene` et `GameEntity`.

Scène La scène correspond à un espace décisionnel : elle traite les événements, consomme les paquets réseau, met à jour les structures de données représentant l'état courant de la partie et déclenche les transitions vers une autre scène si nécessaire. Elle est responsable de la cohérence logique.

Entité L'entité, quant à elle, fait seulement de la représentation. Elle traduit la logique décidée par la scène en éléments graphiques concrets : sprites, tuiles, animations, positions affichées. Elle ne possède pas la responsabilité de déterminer ou est un joueur, ou quels sont les paramètres actuels d'un lobby. Elle reçoit un état déjà validé et le rend visible au joueur.

Une séparation essentielle Cette distinction est essentielle pour plusieurs raisons. D'abord, elle évite le mélange entre logique réseau et logique graphique. Une entité ne dépend pas directement du socket ni des paquets. Elle reçoit un état déjà interprété par la scène.

Ensuite, cette séparation améliore la maintenabilité : modifier le rendu n'implique pas de toucher à la logique réseau, et inversement.

Enfin, elle rend le système plus évolutif. Il serait possible d'adapter le rendu (par exemple changer une animation ou le style graphique) sans impacter la gestion des paquets ou la synchronisation.

4.4 Réseau coté Client : réception

La réception réseau côté client repose sur une architecture volontairement séparée de la boucle principale. Le socket TCP est configuré en mode non bloquant, puis un thread dédié est lancé via `startNetworkReceiver`. Ce thread exécute en continu `recvPacket`, récupère les paquets entrants et les insère dans une file partagée (`m packetQueue`) protégée par mutex. Cette séparation garantit que la boucle principale du `SceneManager` ne sera jamais bloquée par une attente réseau.

Le choix d'un thread distinct permet d'isoler les opérations d'entrée/sortie (I/O) du rendu graphique et du traitement logique. La scène active, dans sa méthode `doUpdate`, consomme ensuite les paquets disponibles via `tryPopPacket`.

Chaque scène décide explicitement quels types de paquets elle traite. Par exemple, `LobbyListScene` interprète `ServerListRooms`, tandis que `GameScene` traite `ServerGameState`, `ServerGameEnd` ou les événements liés au pouvoir de Pac-Man. Ce modèle offre un contrôle précis : une scène ne traite que les messages pertinents à son contexte, aucun autre.

La synchronisation est assurée par un `std : :mutex` protégeant l'accès à la file. Cette précaution évite toute condition de course entre le thread réseau (producteur) et la scène active (consommateur). L'ordonnancement des paquets est préservé naturellement par la structure FIFO de la file, tandis que TCP garantit l'ordre et la fiabilité de livraison.

4.5 Réseau coté Client : envois

Les envois côté client sont volontairement limités et contrôlés. Contrairement à la réception asynchrone, l'émission dépend du contexte et du rythme de jeu. Autrement dit, ce sont les scènes qui décident quand et quoi envoyer. Chaque scène est donc responsable des messages correspondant à son domaine fonctionnel.

Envoi périodique Dans `GameScene` par exemple, un horodatage (`m lastSend`) régule la fréquence des transmissions. L'envoi d'un `ClientMove` n'a lieu que si une touche directionnelle est active et si l'intervalle minimal est respecté. L'objectif est d'éviter un envoi par frame graphique tout en garantissant une fréquence stable compatible avec la logique serveur.

Envoi événementiel Dans le cas du lobby, le fonctionnement est différent. L'entité détecte une interaction (bouton +/-, ready, changement de rôle) et expose cette action via un enum `LobbyAction`. La scène récupère alors l'action qui vient d'être effectuée puis la transforme en message réseau et l'envoi. L'envoi est donc déclenché uniquement quand une action de la part du joueur a lieu.

Principe d'envoi Que ce soit en envoi périodique ou événementiel, le principe d'envoi est toujours le même. Quand on détecte une action utilisateur et/ou une période écoulée, on crée un `gf : :Packet` qu'on encapsule dans une structure créée spécifiquement pour la communication client -> serveur, et on appelle `sendPacket` pour l'envoyer au serveur. Ce sera ensuite au serveur de gérer notre envoi.

C'est uniquement ce qui est reçu par le Client et donc décidé par le Serveur qui sera effectivement affiché. L'envoi permet uniquement de transmettre au serveur une intention du client.

4.6 Input, Actions, et EventTypes

La gestion des entrées repose sur une idée simple : ne pas manipuler directement les touches clavier ou les clics, mais passer par des actions plus claires. Plutôt que de tester partout si une touche est pressée, on définit des actions comme `upAction`, `downAction`, `leftAction` ou `rightAction`, qui représentent directement ce que le joueur veut faire. Cela rend le code plus facile à lire et évite de disperser la gestion des entrées dans toute l'application.

Les événements manipulés côté client sont de plusieurs types. On retrouve d'abord les entrées clavier, utilisées principalement pendant la partie pour les déplacements. Les événements souris sont plutôt utilisés dans les menus et les lobbys, par exemple pour gérer le survol ou les clics sur des boutons. Enfin, certains événements système comme `Resized` ou `Closed` permettent de gérer respectivement l'adaptation de l'affichage et la fermeture propre du client. Chaque type d'événement a donc un rôle différent selon la scène active.

Pour les entrées clavier, le système d'actions permet de simplifier la gestion des touches. Chaque action est liée à une touche via un `Keycode`, puis marquée comme continue avec `setContinuous()`. Cela permet de gérer naturellement le maintien d'une touche : tant qu'elle est enfoncée, l'action reste active et peut être utilisée à chaque mise à jour. On évite ainsi de devoir gérer manuellement les répétitions ou les états des touches, ce qui rend le comportement plus régulier et le code plus simple à maintenir.

Le traitement des événements se ensuite fait au niveau des scènes, dans `doProcessEvent`. Chaque scène récupère l'événement reçu et décide ce qu'elle en fait. Certaines scènes réagissent surtout à la souris, d'autres au clavier, mais toutes peuvent gérer le redimensionnement de manière cohérente. Par exemple, dans `GameScene`, les événements clavier sont transmis au conteneur d'actions via `actions.processEvent(event)`, tandis que l'événement `Resized` déclenche `resizeYourself()` pour recalculer la vue. Cette logique locale évite d'avoir un gestionnaire global qui mélange tous les cas possibles et d'augmenter l'autonomie des scènes car les scènes décident si font quelque chose avec l'événement ou non si l'événement ne la concerne pas.. Tous les événements ne sont cependant pas traités de la même manière du point de vue réseau.

Certains restent purement locaux, comme le mouvement de la souris ou le redimensionnement de la fenêtre, car ils ne modifient pas l'état du jeu côté serveur. À l'inverse, certaines actions issues des entrées utilisateur sont envoyées au serveur, comme les déplacements en jeu ou les interactions dans le lobby (changer un paramètre, se déclarer prêt, etc.). Dans ces cas-là, le client n'envoie pas un état complet mais une intention, et c'est le serveur qui décide de la valider ou non en renvoyant un état mis à jour. Cette distinction permet de limiter les communications réseau tout en gardant un modèle cohérent où le serveur reste responsable de l'état du jeu.

4.7 Rendu des Entités

Maintenant que la logique du jeu et les échanges réseau sont gérés, il reste une dernière étape côté client : afficher l'état du jeu. C'est le rôle des entités (`GameEntity`, `LobbyEntity`, etc.). Chaque scène possède une unique entité, volontairement simple, dont le but est uniquement de traduire un état en éléments graphiques. Autrement dit, l'entité ne prend pas de décision : elle affiche ce que la scène lui donne. Par exemple, dans le lobby, si un joueur est prêt, l'entité affiche "(prêt)", sinon "(pas prêt)". Cette

séparation permet de garder un rendu indépendant de la logique métier.

Dans la plupart des entités, le rendu est organisé en plusieurs fonctions distinctes, chacune responsable d'un élément précis : pour le jeu, on a `renderMap` pour la carte, `renderPacGommes` pour les objets, `renderSprites` pour les joueurs, et d'autres fonctions pour l'interface (timer, points de vie, effets). Ce découpage permet de garder un code lisible et de modifier facilement une partie du rendu sans impacter le reste. La fonction principale `render` orchestre simplement ces appels dans le bon ordre.

Un point important à mentionner concerne le déplacement des joueurs. Le serveur envoie des positions selon un interval de temps choisi, ce qui pourrait produire un mouvement visuellement saccadé si on affichait directement ces positions. Pour éviter cela, une interpolation est utilisée entre la position de départ (`startPos`) et la destination (`destPos`). Le temps attendu entre deux mises à jour (`expectedDuration`) permet de calculer une position intermédiaire fluide à chaque frame. Ce mécanisme, implémenté dans `renderSprites`, permet d'obtenir un rendu visuellement continu même si les mises à jour réseau sont espacées.

Le rendu repose également sur l'utilisation de textures et de spritesheets. Les personnages utilisent des animations basées sur des tilesets (par exemple Pacman avec plusieurs frames selon la direction), tandis que la carte utilise un tileset avec des `RectF` pour sélectionner la bonne portion de texture selon le type de mur. Cela permet de réutiliser une seule image pour représenter de nombreux éléments, ce qui est à la fois plus efficace et plus simple à maintenir.

Le rendu est effectué dans un espace logique fixe (par exemple 1280×720), puis simplement adapté à la fenêtre via le système de vue (`resizeYourself`). Cela permet de garder des proportions cohérentes quel que soit le format d'affichage.

5 Protocoles et échanges réseau

5.1 Type de communication

Les échanges de données entre le client et le serveur s'effectue via des socket, en utilisant les classes fournies par *GamedevFramework*.

GamedevFramework offre le choix entre deux protocoles de transport :

- Le **TCP** avec `gf::TcpSocket` et `gf::TcpListener`
- Le **UDP** avec `gf::UdpSocket`

Pour ce projet, le protocole **TCP** est préféré au protocole **UDP** pour sa fiabilité.

Le serveur contient une socket de connexion **TCP** (`gf::TcpListener`), initialisé en même temps que lui. Cette socket gère les connexions des clients dans un thread, en associant une socket de communication pour chaque client. Le client, de son côté, initialise directement une socket de communication.

5.2 Format des messages

Pour s'échanger les données, le serveur et le client utilise des structure *communes* et des structure de *communication*.

- Les **structure communes** sont des structures contenant des données utilisé à la fois par le client et le serveur. Parmi eux, il y a, par exemple la structure *PlayerData*, contenant les informations relatives à un joueur (position, points de vie, etc...).

```
struct PlayerData
{
    uint32_t id;
    float x, y;
    uint32_t color;
    std::string name;
    PlayerRole role;
    int score;
    bool ready = false;
    unsigned int hp = 0;
};
```

- Les **strucutes de communication** sont les structures sérialisé pour être envoyé, et reçu avant d'être désérialisé avec pour être utilisé. Ces structures contiennent la plupart du temps une ou plusieurs structures *communes*, mais ils ont aussi un champs *type* du type `gf::Id`, afin d'être indentifié lors de la désérialisation.

```
struct ServerListRoomPlayers
{
    static constexpr gf::Id type = "ServerListRoomPlayers"_id;
    std::vector<PlayerData> players;
```

```
};
```

Sérialisation et désérialisation Afin de pouvoir être échangé entre le serveur et le client, les structures doivent pouvoir être sérialisé en suite d'octets avant d'être envoyé, et lors de la réception, être désérialisé pour retrouver leur forme initiale.

Pour cela, un **opérateur** | avec un type *Archive* a été implémenté pour chaque structures envoyé, permettant leur sérialisation et désérialisation grâce aux fonctionnalités du *GamedevFramework*

```
template <typename Archive>
Archive &operator|(Archive &ar, PlayerData &data)
{
    return ar | data.id | data.x | data.y | data.color
           | data.name | data.role | data.score
           | data.ready | data.hp;
}
```

```
template <typename Archive>
Archive &operator|(Archive &ar, ServerListRoomPlayers &data)
{
    return ar | data.players;
}
```

Une fois les opérateurs implémentés, les structures peuvent être empaquetées via la fonction `gf::Packet::is(data)` et dépaquetées avec la fonction `gf::Packet::as<typename>()` pour être envoyées/reçues

5.3 Protocole

Connexion Lors de son exécution, le client cherche à se connecter au serveur à l'adresse IP et au port passés en paramètre. Le serveur, s'il existe, reçoit cette demande, et associe au client un identifiant, sous la forme d'un entier positif, avant de répondre au client avec cette identifiant et la liste des salons disponible. Une fois la réponse du serveur reçu, le client sauvegarde son identifiant, afin de savoir qui il est parmi la liste des joueurs, puis affiche la scène du début du jeu.

Liste des salons Dans la liste des salons, le joueur peut choisir de rejoindre un salon existant, ou un en crée un nouveau. Lorsque l'une des deux options est choisi, le client envoi une demande de création ou de jointe d'un salon au serveur. Le serveur traite cette demande, en ajoutant le joueur dans le salon créé/rejoint, et envoi au client la liste des joueurs et les paramètres du salon. Dans le cas où le joueur rejoint un salon, le serveur envoi aussi à tous les autres clients du salon la nouvelle liste des joueurs pour les tenir à jour. Si le client tente de rejoindre un salon qui n'existe pas ou qui est plein le serveur en crée un nouveau. Une fois la réponse du serveur reçu, le client change de scène (vers la scène du salon), et affiche les informations reçues.

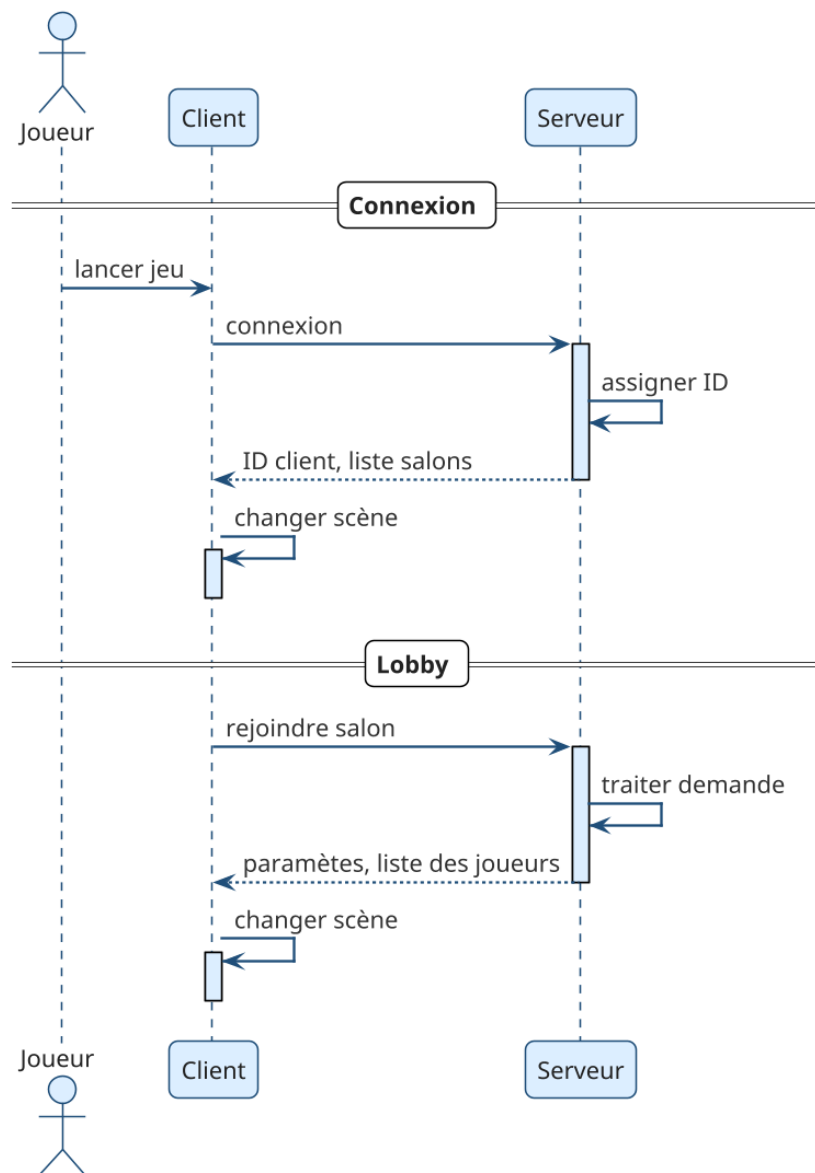


FIGURE 11 – Échanges entre le client et le serveur lors de la connexion/lobby

Salon Dans le salon, le client peut soit changer de rôle, changer de statut (prêt/pas prêt), ou changer les paramètres du salon. Lorsqu'un client veut changer de rôle ou de statut, il envoie une demande au serveur. Le serveur reçoit cette demande, la traite, et informe tous les clients de la nouvelle liste des joueurs. Le client reçoit cette information et l'affiche.

Lorsqu'un client veut changer les paramètres de la partie, il y a d'abord une première vérification de son côté pour réajuster les valeurs s'il elles sont fausses (Ex : Point de vie du Pac-Man à 0). Après cela, il envoie les paramètres souhaités au serveur. Le serveur reçoit les paramètres, et vérifie à son tour la validité des valeurs. Si elles sont bonnes, le serveur envoie à tous les clients du salon les nouveaux paramètres, ou sinon, les paramètres valides avant la demande du client. Le client reçoit la réponse et affiche les paramètres.

Lorsque tous les clients sont prêts, le serveur vérifie si leurs rôles sont valides (s'il y a un

Pac-man). Si c'est le cas, le serveur génère les données du jeu (Plateau, Joueurs), et informe les clients du début de la partie. Les clients reçoivent le signal de début de partie et change de scène (vers la scène de jeu).

Dans le cas où les rôles ne sont pas valide, le serveur envoie aux clients l'erreur, et les clients l'affichent.

En jeu En jeu, l'unique action que peut réaliser le client est le déplacement de son personnage. Pour chaque déplacement, le client envoie une demande au serveur. Le serveur reçoit cette demande, et vérifie si la position souhaitée par le client est valide (Pas un mur). Si c'est le cas, le serveur traite ensuite les conséquences du déplacement, notamment si le personnage entre en collision avec un autre. Après cela, le serveur envoie à tous les clients l'état du jeu.

Le client reçoit l'état du jeu, et l'affiche

Le serveur envoie aussi périodiquement aux clients le temps restant, afin de les tenir informés

Lorsqu'une des conditions de fin de jeu est atteinte (Temps écoulé, Le Pac-Man n'a plus de vie, les fantômes n'ont plus de vie, ou toutes les pacgomes ont été mangées), le serveur informe tous les clients, en leur précisant la raison, et met fin à la partie

En recevant le message de fin, le client change de scène, et affiche la raison de la fin du jeu.

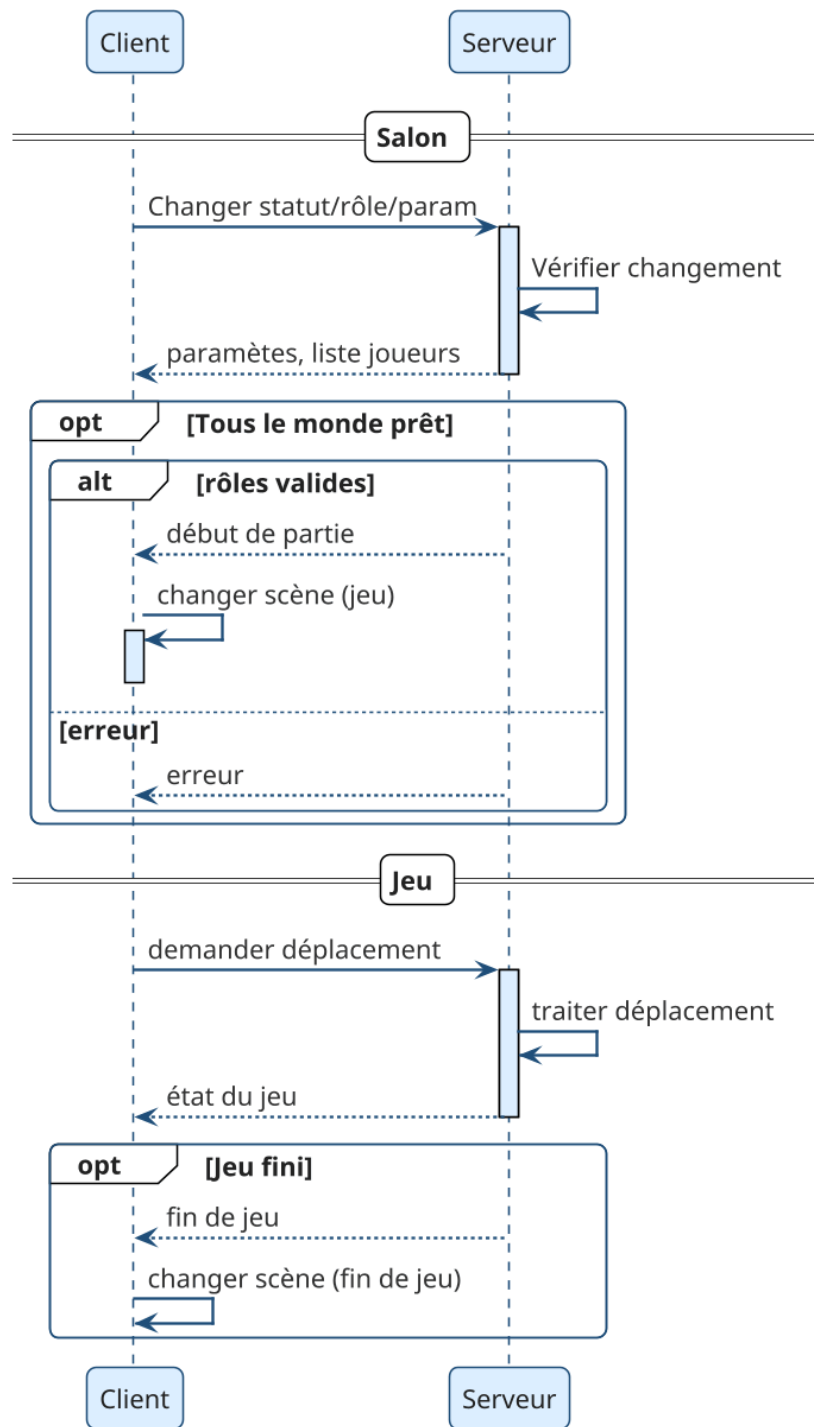


FIGURE 12 – Échanges entre le client et le serveur lors dans le salon/en jeu

6 Bilan et perspectives

6.1 Compétences acquises

La réalisation de ce projet nous a permis de mobiliser et d’approfondir plusieurs compétences acquises au cours de notre formation.

Sur le plan technique, nous avons développé des compétences en programmation réseau, notamment dans la conception d’une architecture client-serveur autoritaire, la gestion des communications et la synchronisation d’un système en temps réel.

Le projet a également renforcé nos compétences en architecture logicielle. La transformation d’un prototype vers une structure plus complexes (Server, Lobby, Room, Game) nous a confrontés à des enjeux de séparation des responsabilités, de maintenabilité et d’évolutivité du code. Cette phase de conception a nécessité une réflexion approfondie sur l’organisation des composants et leurs interactions.

Enfin, le travail de conception globale du système, la génération procédurale du plateau, l’implémentation d’une intelligence artificielle, la définition des règles de synchronisation, nous a permis de développer une approche méthodique dans l’analyse des besoins, la modélisation et la résolution de problèmes complexes.

6.2 Résultat final

Le projet aboutit à un jeu multijoueur fonctionnel capable de gérer plusieurs connexions simultanées. Les principales fonctionnalités prévues ont été implémentées : gestion des lobbys et des parties, synchronisation en temps réel des joueurs, génération procédurale du labyrinthe, ainsi qu’un système d’intelligence artificielle pour les entités ennemies.

La cohérence du jeu est assurée par la mise à jour périodique côté serveur, ce qui garantit que les états partagés aux clients sont bien valides. Les collisions, le déplacement et les interactions ont été validés et intégrés de manière stable.

6.3 Améliorations possibles

Plusieurs axes d’évolution peuvent être envisagés pour enrichir le gameplay et la rejouabilité.

Tout d’abord, introduire des capacités spécifiques aux rôles, telles que : vitesse, invisibilité, téléportation, traversée de murs, invulnérabilité, ou effets sur les ennemis (paralyse, aveuglement, localisation). Ces pouvoirs permettraient des stratégies variées et des interactions plus riches entre joueurs.

Le labyrinthe pourrait inclure plusieurs étages, des salles secrètes, des passages modifiables en cours de partie ou des portails supplémentaires. Ces évolutions augmenteraient la complexité et la diversité des parties.

De plus, Pac-Man pourrait accomplir des missions ou tâches déclenchant des alertes pour les fantômes. Des rôles secrets (traître parmi les fantômes, classe spéciale) pourraient introduire des mécanismes de coopération ou de compétition inattendus.

Conclusion

Ce projet a permis de mettre en pratique les connaissances acquises durant notre formation, notamment en développement réseau, de la programmation multithread et de la conception logicielle et objet.

La création d'un jeu multijoueur inspiré de Pac-Man a nous a permis d'explorer des problématiques telles que la synchronisation des clients, la gestion de plusieurs parties simultanées et la programmation d'une intelligence artificielle pour des entités ennemies.

L'architecture choisi a posé une base solide pour d'éventuelles extensions, comme l'introduction de pouvoirs spéciaux, de rôles secrets ou de labyrinthes multi-étages.

Ainsi, ce projet constitue à la fois un aboutissement des compétences acquises et un terrain d'expérimentation pour de futures évolutions, offrant de nombreuses perspectives d'amélioration et d'enrichissement du gameplay.

Sitographie

Références

- [1] Gamedev Framework (gf) 1.2.0, <https://gamedevframework.github.io/v1.2.0/>, consulté le 26 février 2026.
- [2] Youtube, J'ai codé une simulation de 869 fourmis, de Code Boy, https://youtu.be/M9ydYnN9_uc?si=7sbFe6Xd0yjbXN1, consulté le 26 février 2026.

Résumé

Le projet de la Licence 3 Informatique a été pour nous l'occasion de concevoir et développer un jeu multijoueur inspiré de Pac-Man, en mettant l'accent sur les problématiques de synchronisation réseau, de génération procédurale et d'intelligence artificielle.

L'objectif principal a été de concevoir une architecture client-serveur autoritaire capable de gérer plusieurs connexions simultanées tout en garantissant la cohérence globale du jeu. Le serveur a été structuré de manière modulaire afin de séparer la gestion réseau, l'organisation des parties et la logique métier.

Le projet intègre également une génération procédurale de labyrinthe basée sur un algorithme de type Prim adapté aux contraintes du gameplay. L'intelligence artificielle des entités ennemies repose sur une approche hybride combinant recherche de chemin et mécanismes inspirés des fourmis.

Le développement a été réalisé principalement en C++ avec le Framework Gamedev ainsi que l'utilisation d'outils de gestion de version et de suivi de projet afin d'assurer une organisation structurée du travail.

Mots-clés

C++ - PacMan - Gamedev Framework - Réseau - IA

Abstract

The third-year Computer Science project provided us with the opportunity to design and develop a multiplayer game inspired by Pac-Man, with a particular focus on network synchronization, procedural generation, and artificial intelligence.

The main objective was to design an authoritative client-server architecture capable of handling multiple simultaneous connections while ensuring the overall consistency of the game state. The server was structured in a modular way in order to separate network management, game session organization, and core game logic.

The project also integrates procedural maze generation based on a Prim-like algorithm adapted to gameplay constraints. The artificial intelligence of enemy entities relies on a hybrid approach combining pathfinding techniques and mechanisms inspired by ant colony behavior.

The development was carried out primarily in C++ using the Gamedev framework, along with version control and project management tools to ensure a structured and organized workflow.

Key-Words

C++ - PacMan - Gamedev Framework - Network - AI